

1. Simple Even Parity

Question:

Find the even parity bit for the data: 1010110

Solution:

Count number of 1s = 4 (even)

- ☐ For even parity → total 1s must remain even
- ☐ Parity bit = **0**

Answer: 10101100

2. Simple Odd Parity

Question:

Find the odd parity bit for the data: 1100101

Solution:

Number of 1s = 4 (even)

☐ For odd parity → need odd number of 1s

☐ Add 1

Answer: 11001011

3. Error Detection (Even Parity)

Question:

Received data: 10110101 (last bit is parity bit, even parity)

Check if error exists.

Solution:

Count 1s = 5 (odd)

☐ Even parity expected → should be even

☐ Error detected

4. Error Detection (Odd Parity)

Question:

Received data: 11100011 (odd parity used). Is there any error??

Solution:

Count 1s = 5 (odd)

☐ Matches odd parity

☐ No error

5. Find Missing Bit

Question:

Data transmitted using even parity: 101?110

Find missing bit.

Solution:

Count known 1s = 4

☐ Already even → missing bit must be 0

Answer: 1010110

2D parity Questions

Question:

Data block:

Parity

1 0 1 ?

0 1 1 ?

1 1 0 ?

? ? ?

Use even parity. Find row and column parity bits.

Solution:

Row parity:

- Row1: $1+0+1 = 2 \rightarrow \text{even} \rightarrow 0$
- Row2: $0+1+1 = 2 \rightarrow \text{even} \rightarrow 0$
- Row3: $1+1+0 = 2 \rightarrow \text{even} \rightarrow 0$

Column parity:

- Col1: $1+0+1 = 2 \rightarrow 0$
- Col2: $0+1+1 = 2 \rightarrow 0$
- Col3: $1+1+0 = 2 \rightarrow 0$

Final parity bits: all 0

Error Detection in 2D Parity

Question:

Received block (even parity):

	1		0		1		0	
	0		1		1		0	
	1		1		1		0	
	0		0		0			

Find error location.

Solution:

Check rows:

- Row3 $\rightarrow 1+1+1 = 3$ (odd) $\square$

Check columns:

- Column3 $\rightarrow 1+1+1 = 3$ (odd) $\square$

$\square$ Error at intersection $\rightarrow$ **Row3, Column3**

Multiple Error Case

Question:

Explain why parity fails for data:

Original: 1010101

Received: 1111101 (even parity)

Solution:

Two bits flipped $\rightarrow$ parity remains same

☐ Parity cannot detect **even number of errors**

Parity Bit Position**Question:**

Data: 1001101

Add odd parity at the beginning.

Solution:

1s = 4 (even) $\rightarrow$ need odd $\rightarrow$ add 1

Answer: 11001101

1.

What is the even parity bit for data 1011001?

A) 0

B) 1

C) Depends on position

D) Cannot be determined

☐ **Answer:** B) 1

Solution: Number of 1s = 4 (even) → already even → parity bit = 0? Wait carefully:
Actually 1s = 4 → even → parity bit = **0**

☐ **Correct Answer:** A) 0

2.

Odd parity bit for 1100110 is:

A) 0

B) 1

C) 2

D) None

☐ **Answer:** B) 1

Solution: 1s = 4 (even) → need odd → add 1

3.

Parity bit is used for:

- A) Error correction
- B) **Error detection**
- C) Data compression
- D) Encryption

☐ **Answer:** B) Error detection

4.

Received data: 10101100 (even parity). What is the result?

- A) Error
- B) No error
- C) Cannot detect
- D) Data corrupted

☐ **Answer:** B) No error

Solution: $1s = 4 \rightarrow \text{even} \rightarrow \text{correct}$

5.

Which error cannot be detected by parity checking?

- A) Single-bit error
- B) Odd number of errors
- C) Even number of errors
- D) Burst error

☐ **Answer:** C) Even number of errors

6.

Parity checking is implemented at which layer?

- A) Network layer
- B) Transport layer
- C) Data Link layer
- D) Application layer

☐ **Answer:** C) Data Link layer

7.

If a single bit flips during transmission, parity checking will:

- A) Always detect
- B) Never detect
- C) Sometimes detect
- D) Correct the error

☐ **Answer:** A) Always detect

8.

In 2D parity, an error is located at:

- A) Any random bit
- B) Intersection of incorrect row and column
- C) Only rows
- D) Only columns

☐ **Answer:** B) Intersection of incorrect row and column

9.

How many errors can 2D parity detect?

- A) Only 1
- B) Only 2
- C) Multiple (but not all)
- D) None

☐ **Answer:** C) Multiple (but not all)

10.

Data 11100011 is transmitted with odd parity(last bit). Received as 11100010. What happened?

- A) No error
- B) One-bit error
- C) Two-bit error
- D) Cannot detect

☐ **Answer:** B) One-bit error

Solution: Parity changes → odd becomes even → detected

11.

Minimum number of parity bits required in simple parity checking:

A) 0

B) 1

C) 2

D) Depends on data size

☐ **Answer:** B) 1

12.

Which statement is correct?

A) Parity can correct errors

B) Parity detects all errors

C) Parity detects only odd errors

D) Parity detects only burst errors

☐ **Answer:** C) Parity detects only odd errors

13.

Even parity is used. Data received: 11010111. What is the status?

- A) Error
- B) No error
- C) Correctable error
- D) Unknown

☐ **Answer:** A) Error

Solution: $1s = 6 \rightarrow \text{even} \rightarrow \text{wait carefully}$

Count: $1+1+0+1+0+1+1+1 = 6$ (even) $\rightarrow$ correct

☐ **Correct Answer:** B) No error

14.

Find missing bit in odd parity: 1011?01

- A) 0
- B) 1
- C) 2
- D) Cannot determine

☐ **Answer:**

Solution: Current $1s = 4 \rightarrow \text{need odd} \rightarrow \text{add } 1 \rightarrow \text{actually missing bit} = 1$

☐ **Correct Answer:** B) 1

Q1. Basic Checksum Calculation

Question:

Find the checksum for the following 8-bit data:

10101010, 11001100, 11110000

Solution:

Step 1: Add all numbers (binary addition)

```
  10101010
+ 11001100
-----
  01110110 (9-bit result, carry generated)
    1
-----
  01110111
```

Add third:

```
  01110111
+ 11110000
-----
  10110011
```

01110011 + 1 = 01101000 (adding carry)

Step 3: Take 1's complement

01101000 → 10010111

□ **Checksum** = 10010111

Q2. Check Receiver Validity

Question:

Data received:

10101010, 11001100, 11110000, Checksum = 11000100

Verify correctness.

Q3. 4-bit Checksum

Question:

Find checksum using 4-bit words:

1011, 0101, 1110

Solution:

Step 1: Add

$1011 + 0101 = 10000$ (carry)

Wrap around:

$0000 + 1 = 0001$

Add next:

$0001 + 1110 = 1111$

Step 2: 1's complement

$1111 \rightarrow 0000$

□ **Checksum = 0000**

Q4. Error Detection

Question:

Received data (4-bit):

1011, 0101, 1110, checksum = 0001

Check if error exists.

Solution:

Add all:

```
1011 + 0101 = 10000 → wrap → 0001
0001 + 1110 = 1111
1111 + 0001 = 10000 → wrap → 0001
```

Final result $\neq$ 1111

☐ Error detected

Q5.

Question:

Find checksum for 8-bit data:

10011001, 01100110, 11111100

Solution:

Step 1: Add first two:

```
10011001
01100110
-----
11111111
```

Step 2: Add third:

```
11111111
11111100
-----
1 1111011
```

Wrap around:

11111011 + 1 = 11111100

Step 3: 1's complement:

11111100 → 00000011

□ **Checksum = 00000011**

Q1. Basic CRC Calculation

Question:

Data = 1101011011

Generator Polynomial = $G(x) = x^4 + x + 1$

Find the CRC codeword.

Solution:

Polynomial $\rightarrow$ Binary: 10011

Append 4 zeros $\rightarrow$ 1101011011 0000

After modulo-2 division:

Remainder = 1110

Final Codeword =

1101011011 1110

Q2. CRC Verification

Question:

Received data = 1011101011010110

Generator Polynomial = $G(x) = x^3 + x^2 + 1$

Check whether error exists.

Solution:

Polynomial $\rightarrow$ Binary: 1101

Perform division:

Remainder = 000

Conclusion:

✓ No error

Q3. CRC with Small Data

Question:

Data = 10101

Generator Polynomial = $G(x) = x^3 + x + 1$

Find CRC bits and transmitted frame.

Solution:

Polynomial $\rightarrow$ Binary: 1011

Append 3 zeros $\rightarrow$ 10101 000

Remainder = 111

Transmitted Frame =

10101 111

Q4. Error Detection Case

Question:

Transmitted data = 11001100

Generator Polynomial = $G(x) = x^3 + x + 1$

CRC = 010

Received data = 11001100010

Check if error occurred.

Solution:

Polynomial $\rightarrow$ Binary: 1011

Perform division:

Remainder $\neq$ 000

Conclusion:

☐ Error detected

Q5. Concept + Calculation

Question:

Data = 111000

Generator Polynomial = $G(x) = x^3 + x^2 + 1$

1. Find CRC
2. If received remainder is 001, what does it indicate?

Solution:

Polynomial $\rightarrow$ Binary: 1101

Append zeros $\rightarrow$ 111000 000

Remainder = 011

Codeword =

111000 011

If remainder = 001 $\neq$ 000

Conclusion:

☐ Error detected